



AgroAPI Ecuador

Tutorial Completo de API REST
con Node.js, Express y JavaScript

Gestion de Cultivos Agricolas

Para principiantes absolutos en APIs web

Guayaquil, Ecuador • 2025

JSON • XML • WebServices • fetch() • PostgreSQL

Índice

1. Que es una API y por que la necesitamos?	2
1.1. Arquitectura del sistema	2
1.2. Verbos HTTP y su significado	2
2. Estructura del Proyecto	3
3. Bases de Datos: JSON y XML	3
3.1. Por que JSON y XML?	3
3.2. Archivo data/cultivos.json	4
3.3. Archivo data/cultivos.xml	5
4. El Servidor: Node.js + Express	5
4.1. Flujo de una peticion HTTP	6
4.2. Archivo server.js – Punto de entrada	6
5. Rutas CRUD: cultivosRoutes.js	7
6. Tabla de Endpoints REST	9
7. Como Probar los WebServices	9
7.1. Opcion A: con curl (terminal)	9
7.1.1. Peticiones GET	9
7.1.2. Peticion POST – Crear nuevo cultivo	10
7.1.3. Peticion PUT – Actualizar estado	10
7.1.4. Peticion DELETE – Eliminar	11
7.2. Opcion B: con Postman o Thunder Client	11
8. Consumir la API desde JavaScript	11
8.1. La funcion fetch()	11
8.2. Funcion central de consumo de API	11
8.3. Leer y mostrar cultivos	12
8.4. Crear, editar y eliminar	13
9. Migracion a PostgreSQL	13
10. Resumen y Proximos Pasos	14
10.1. Lo que aprendiste en este tutorial	14
10.2. Proximos pasos sugeridos	15

1. Que es una API y por que la necesitamos?

Una **API** (Application Programming Interface) es un conjunto de reglas que permite que dos programas se comuniquen entre si. En el contexto web, una **API REST** expone datos a traves de URLs y responde en formato JSON.

► Nota

Piensa en una API como un mesero: tu (el cliente) le dices que quieres (petición HTTP), el mesero va a la cocina (servidor) y te trae la respuesta (datos JSON). No necesitas saber como funciona la cocina.

1.1. Arquitectura del sistema

La **Figura 1** muestra las tres capas del sistema y como se comunican entre si. El flujo siempre va del cliente hacia el servidor y regresa como respuesta JSON.

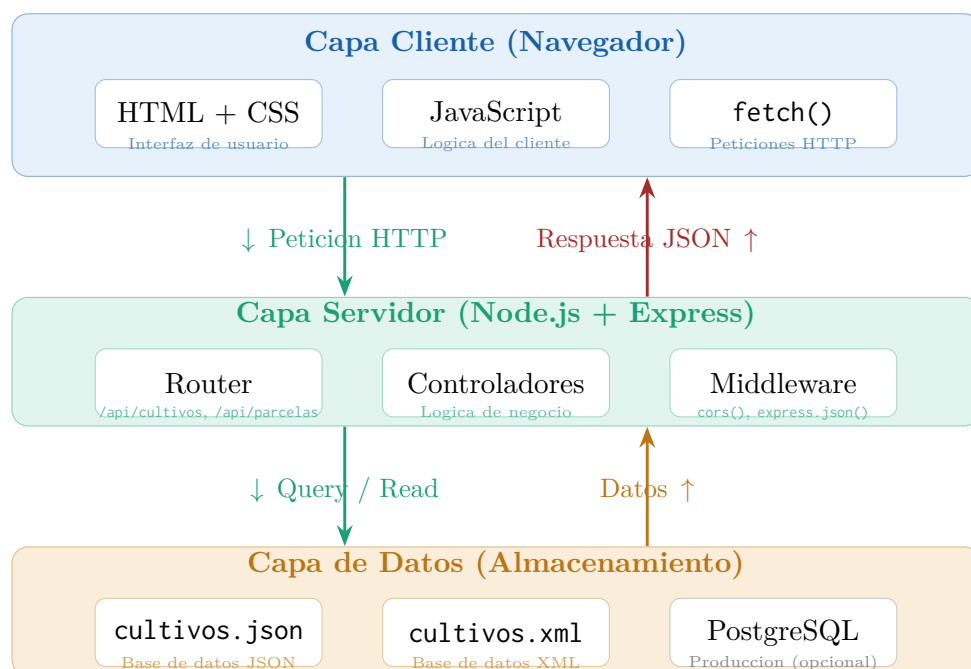


Figura 1: Arquitectura de tres capas de la aplicacion AgroAPI.

1.2. Verbos HTTP y su significado

Las operaciones CRUD se mapean directamente a verbos HTTP. La **Tabla 1** resume la correspondencia.

2. Estructura del Proyecto

Verbo	Operacion	Que hace	Ejemplo
GET	Read (Leer)	Obtener datos	GET /api/cultivos
POST	Create (Crear)	Insertar nuevo dato	POST /api/cultivos
PUT	Update (Actualizar)	Modificar un dato	PUT /api/cultivos/3
DEL	Delete (Eliminar)	Borrar un dato	DELETE /api/cultivos/3

Cuadro 1: Verbos HTTP y su correspondencia con operaciones CRUD.

Paso 1: Crear las carpetas

```

1 mkdir cultivos-api
2 cd cultivos-api
3 mkdir data routes cliente

```

La **Figura 2** muestra el arbol de directorios del proyecto. Cada carpeta tiene una responsabilidad clara: **data/** almacena los archivos JSON y XML, **routes/** contiene las rutas de la API, y **cliente/** guarda la aplicacion web.

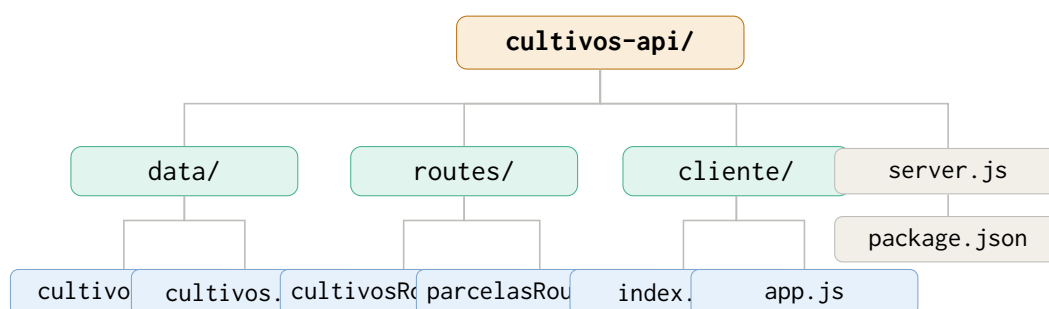


Figura 2: Arbol de directorios del proyecto cultivos-api.

3. Bases de Datos: JSON y XML

3.1. Por que JSON y XML?

Para aprender usamos archivos en lugar de una base de datos real. Node.js puede leer ambos formatos directamente. La **Tabla 2** compara sus características principales.

	JSON	XML
Sintaxis	Llaves {} y corchetes []	Etiquetas <tag>...</tag>
Nativo en JS	✓ Si (JSON.parse())	No. Requiere xml2js
Peso	Mas ligero	Mas verboso
Uso actual	APIs REST modernas	Legado, SOAP, configs

Cuadro 2: Comparativa JSON vs XML.

3.2. Archivo data/cultivos.json

```
1 {
2   "cultivos": [
3     {
4       "id": 1,
5       "nombre": "Maiz Amarillo",
6       "variedad": "Hibrido DK-7088",
7       "fecha_siembra": "2024-11-15",
8       "fecha_cosecha_estimada": "2025-04-20",
9       "estado": "en_crecimiento",
10      "parcela_id": 1,
11      "area_hectareas": 3.5,
12      "rendimiento_esperado_qq": 140,
13      "observaciones": "Buen desarrollo vegetativo"
14    },
15    {
16      "id": 2,
17      "nombre": "Arroz",
18      "variedad": "INIA Sequia",
19      "fecha_siembra": "2024-12-01",
20      "estado": "germinacion",
21      "parcela_id": 2,
22      "area_hectareas": 5.0
23    }
24  ],
25  "parcelas": [
26    {
27      "id": 1,
28      "nombre": "Parcela Norte A",
29      "ubicacion": "Km 24 via Daule",
30      "area_total_ha": 5.0,
31      "tipo_suelo": "Franco arcilloso"
32    }
33  ],
34  "insumos": [
35    {
36      "id": 1,
37      "nombre": "Urea 46%",
38      "tipo": "fertilizante",
39      "unidad": "kg",
40      "stock": 500,
41      "precio_unitario": 0.75
42    }
43  ]
44 }
```

```
43   ]  
44 }
```

3.3. Archivo data/cultivos.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>  
2 <agro_sistema>  
3  
4   <cultivos>  
5     <cultivo id="1">  
6       <nombre>Maiz Amarillo</nombre>  
7       <variedad>Hibrido DK-7088</variedad>  
8       <fecha_siembra>2024-11-15</fecha_siembra>  
9       <estado>en_crecimiento</estado>  
10      <parcela_id>1</parcela_id>  
11      <area_hectareas>3.5</area_hectareas>  
12    </cultivo>  
13    <cultivo id="2">  
14      <nombre>Arroz</nombre>  
15      <variedad>INIA Sequia</variedad>  
16      <fecha_siembra>2024-12-01</fecha_siembra>  
17      <estado>germinacion</estado>  
18      <parcela_id>2</parcela_id>  
19      <area_hectareas>5.0</area_hectareas>  
20    </cultivo>  
21  </cultivos>  
22  
23  <parcelas>  
24    <parcela id="1">  
25      <nombre>Parcela Norte A</nombre>  
26      <ubicacion>Km 24 via Daule</ubicacion>  
27      <area_total_ha>5.0</area_total_ha>  
28      <tipo_suelo>Franco arcilloso</tipo_suelo>  
29    </parcela>  
30  </parcelas>  
31  
32 </agro_sistema>
```

4. El Servidor: Node.js + Express

Paso 2: Instalar dependencias

```
1 # Crear el package.json  
2 npm init -y  
3  
4 # Instalar Express (servidor web) y CORS (seguridad)  
5 npm install express cors xml2js  
6  
7 # Instalar nodemon para desarrollo (recarga automatica)  
8 npm install --save-dev nodemon
```

4.1. Flujo de una petición HTTP

La **Figura 3** ilustra el recorrido completo de una petición desde el navegador hasta la base de datos y de regreso. Los numeros indican el orden de ejecucion: la petición viaja de izquierda a derecha (pasos 1–4) y la respuesta regresa al navegador (paso 5).

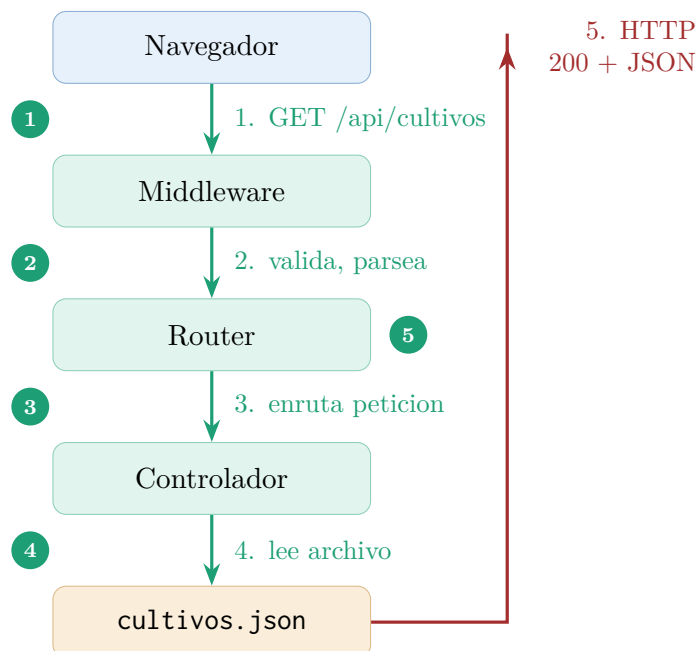


Figura 3: Flujo completo de una petición HTTP a través de las capas del servidor.

4.2. Archivo `server.js` – Punto de entrada

```

1 // server.js -- Punto de entrada del WebService
2 const express = require('express');
3 const cors    = require('cors');
4 const path    = require('path');
5
6 // -- PARA POSTGRESQL: descomenta estas líneas -----
7 // const { Pool } = require('pg');
8 // const pool = new Pool({
9 //   host: 'localhost', port: 5432,
10 //   database: 'agro_db',
11 //   user: 'postgres', password: 'tu_password'
12 // });
13 // module.exports = { pool };
14 // -----
15
16 const app = express();
17 const PORT = 3000;
18
19 // -- MIDDLEWARE -----
20 app.use(cors()); // Permite peticiones desde cualquier origen
21 app.use(express.json()); // Parsea el body JSON de POST/PUT
22 app.use(express.static(path.join(__dirname, 'cliente')));
23
24 // -- RUTAS -----

```

```

25 app.use('/api/cultivos', require('./routes/cultivosRoutes'));
26 app.use('/api/parcelas', require('./routes/parcelasRoutes'));
27 app.use('/api/insumos', require('./routes/insumosRoutes'));
28
29 // -- INFO DE LA API -----
30 app.get('/api', (req, res) => {
31   res.json({
32     mensaje: 'Bienvenido a AgroAPI Ecuador',
33     version: '1.0.0',
34     endpoints: {
35       cultivos: 'GET|POST /api/cultivos',
36       cultivo: 'GET|PUT|DELETE /api/cultivos/:id',
37       parcelas: 'GET|POST /api/parcelas',
38       insumos: 'GET|POST /api/insumos'
39     }
40   });
41 });
42
43 // -- MANEJO DE ERRORES -----
44 app.use((req, res) => {
45   res.status(404).json({ error: 'Ruta no encontrada' });
46 });
47
48 // -- INICIAR -----
49 app.listen(PORT, () => {
50   console.log('Servidor en http://localhost:' + PORT);
51 });

```

5. Rutas CRUD: cultivosRoutes.js

El siguiente código implementa los cuatro verbos HTTP para el recurso **cultivos**. Los comentarios marcados con **CON POSTGRESQL** muestran como adaptar cada método si se migra a una base de datos relacional.

```

1 // routes/cultivosRoutes.js
2 const express = require('express');
3 const router = express.Router();
4 const fs = require('fs');
5 const path = require('path');
6
7 const DB_PATH = path.join(__dirname, '../data/cultivos.json');
8
9 function leerDatos() {
10   return JSON.parse(fs.readFileSync(DB_PATH, 'utf-8'));
11 }
12 function guardarDatos(d) {
13   fs.writeFileSync(DB_PATH, JSON.stringify(d, null, 2));
14 }
15
16 // -- GET /api/cultivos (lista completa, filtro por estado) --
17 router.get('/', (req, res) => {
18   try {
19     const datos = leerDatos();
20     let cultivos = datos.cultivos;

```



```

21     if (req.query.estado)
22         cultivos = cultivos.filter(c => c.estado === req.query.estado);
23     res.json({ total: cultivos.length, cultivos });
24     // CON POSTGRESQL:
25     // const r = await pool.query('SELECT * FROM cultivos');
26     // res.json({ total: r.rowCount, cultivos: r.rows });
27 } catch (e) { res.status(500).json({ error: e.message }); }
28 });
29
30 // -- GET /api/cultivos/:id -----
31 router.get('/:id', (req, res) => {
32     try {
33         const id = parseInt(req.params.id);
34         const cultivo = leerDatos().cultivos.find(c => c.id === id);
35         if (!cultivo)
36             return res.status(404).json({ error: 'Cultivo no encontrado' });
37         res.json(cultivo);
38     } catch (e) { res.status(500).json({ error: e.message }); }
39 });
40
41 // -- POST /api/cultivos -----
42 router.post('/', (req, res) => {
43     try {
44         const { nombre, variedad, fecha_siembra, parcela_id } = req.body;
45         if (!nombre || !variedad || !fecha_siembra || !parcela_id)
46             return res.status(400).json({ error: 'Faltan campos obligatorios' });
47         const datos = leerDatos();
48         const nuevoId = Math.max(...datos.cultivos.map(c => c.id)) + 1;
49         const nuevo = {
50             id: nuevoId, nombre, variedad, fecha_siembra,
51             estado: req.body.estado || 'germinacion',
52             parcela_id: parseInt(parcela_id),
53             area_hectareas: parseFloat(req.body.area_hectareas) || 0
54         };
55         datos.cultivos.push(nuevo);
56         guardarDatos(datos);
57         res.status(201).json({ mensaje: 'Cultivo creado', cultivo: nuevo });
58         // CON POSTGRESQL:
59         // const r = await pool.query(
60         //     'INSERT INTO cultivos(...) VALUES($1,...) RETURNING *',
61         //     [nombre, variedad, ...]
62         // );
63         // res.status(201).json({ cultivo: r.rows[0] });
64     } catch (e) { res.status(500).json({ error: e.message }); }
65 });
66
67 // -- PUT /api/cultivos/:id -----
68 router.put('/:id', (req, res) => {
69     try {
70         const id = parseInt(req.params.id);
71         const datos = leerDatos();
72         const idx = datos.cultivos.findIndex(c => c.id === id);
73         if (idx === -1)
74             return res.status(404).json({ error: 'Cultivo no encontrado' });
75         datos.cultivos[idx] = { ...datos.cultivos[idx], ...req.body, id };
76         guardarDatos(datos);
77         res.json({ mensaje: 'Actualizado', cultivo: datos.cultivos[idx] });

```

```
78   } catch (e) { res.status(500).json({ error: e.message }); }
79 });
80
81 // -- DELETE /api/cultivos/:id -----
82 router.delete('/:id', (req, res) => {
83   try {
84     const id    = parseInt(req.params.id);
85     const datos = leerDatos();
86     const idx   = datos.cultivos.findIndex(c => c.id === id);
87     if (idx === -1)
88       return res.status(404).json({ error: 'Cultivo no encontrado' });
89     const eliminado = datos.cultivos.splice(idx, 1)[0];
90     guardarDatos(datos);
91     res.json({ mensaje: 'Eliminado', cultivo_eliminado: eliminado });
92   } catch (e) { res.status(500).json({ error: e.message }); }
93 });
94
95 module.exports = router;
```

6. Tabla de Endpoints REST

La **Tabla 3** lista todos los endpoints disponibles en la API. Los metodos GET no requieren body; POST y PUT reciben un objeto JSON en el cuerpo de la peticion.

7. Como Probar los WebServices

7.1. Opcion A: con curl (terminal)

Paso 3: Iniciar el servidor y probar

```
1 # Terminal 1: iniciar el servidor
2 node server.js
3
4 # Terminal 2: probar los endpoints con curl
```

7.1.1. Peticiones GET

```
1 # Listar todos los cultivos
2 curl http://localhost:3000/api/cultivos
3
4 # Filtrar por estado
5 curl "http://localhost:3000/api/cultivos?estado=en_crecimiento"
6
7 # Obtener el cultivo con ID 1
8 curl http://localhost:3000/api/cultivos/1
9
10 # Ver todas las parcelas
11 curl http://localhost:3000/api/parcelas
```

Metodo	URL	Descripcion	Parametros
GET	/api	Info de la API	—
GET	/api/cultivos	Listar todos	?estado=...
GET	/api/cultivos/:id	Obtener uno	:id
POST	/api/cultivos	Crear nuevo	Body JSON
PUT	/api/cultivos/:id	Actualizar	:id + Body
DEL	/api/cultivos/:id	Eliminar	:id
GET	/api/parcelas	Listar parcelas	—
GET	/api/parcelas/:id	Obtener una	:id
POST	/api/parcelas	Crear parcela	Body JSON
PUT	/api/parcelas/:id	Actualizar	:id + Body
DEL	/api/parcelas/:id	Eliminar	:id
GET	/api/insumos	Listar insumos	?tipo=...
POST	/api/insumos	Crear insumo	Body JSON
PUT	/api/insumos/:id	Actualizar	:id + Body
DEL	/api/insumos/:id	Eliminar	:id

Cuadro 3: Endpoints REST completos de la AgroAPI.

7.1.2. Peticion POST – Crear nuevo cultivo

```

1 curl -X POST http://localhost:3000/api/cultivos \
2   -H "Content-Type: application/json" \
3   -d '{
4     "nombre":      "Pimiento",
5     "variedad":    "California Wonder",
6     "fecha_siembra": "2025-01-20",
7     "parcela_id":  5,
8     "area_hectareas": 0.5,
9     "estado":      "germinacion"
10  }'
```

7.1.3. Peticion PUT – Actualizar estado

```

1 curl -X PUT http://localhost:3000/api/cultivos/2 \
2   -H "Content-Type: application/json" \
3   -d '{
4     "estado":      "en_crecimiento",
5     "observaciones": "Malezas controladas, buen desarrollo"
6   }'
```

7.1.4. Peticion DELETE – Eliminar

```
1 curl -X DELETE http://localhost:3000/api/cultivos/6
```

7.2. Opcion B: con Postman o Thunder Client

1. Instala **Thunder Client** como extension de VS Code o descarga **Postman**.
2. Crea una nueva peticion.
3. Selecciona el metodo (GET, POST, etc.) y escribe la URL completa.
4. Para POST y PUT: ve a *Body* → *JSON* y escribe el objeto.
5. Haz clic en *Send* y observa la respuesta.

► Nota

Los codigos de respuesta HTTP mas importantes son: **200 OK** (exito), **201 Created** (recurso creado), **400 Bad Request** (datos incorrectos), **404 Not Found** (no encontrado), **500 Server Error** (error del servidor).

8. Consumir la API desde JavaScript

8.1. La funcion fetch()

`fetch()` es la funcion nativa del navegador para hacer peticiones HTTP. Devuelve una **Promesa** (Promise): la respuesta llega de forma asincrona. La **Figura 4** muestra el encadenamiento de promesas desde la llamada hasta la actualizacion del DOM.

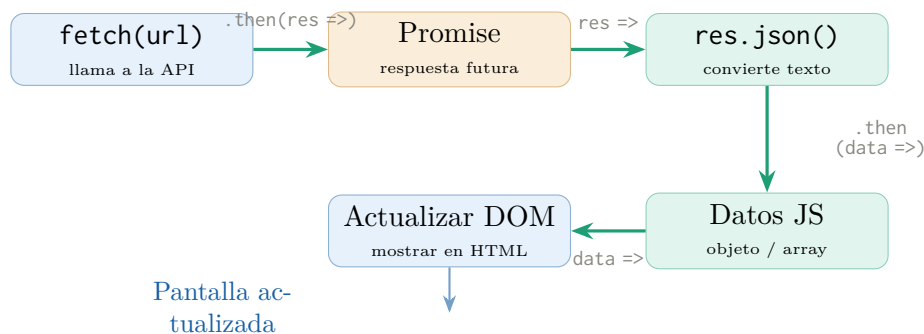


Figura 4: Cadena de promesas de `fetch()`: desde la peticion hasta la actualizacion del DOM.

8.2. Funcion central de consumo de API

La funcion `pedirAPI()` encapsula toda la logica de comunicacion. Internamente usa `fetch()` siguiendo el flujo de la **Figura 4**.

```
1 // cliente/app.js
2 const BASE_URL = 'http://localhost:3000/api';
3
4 /**
5  * Funcion central para hacer peticiones a la API.
6  * url      : endpoint, ej: BASE_URL + '/cultivos'
7  * metodo   : 'GET', 'POST', 'PUT', 'DELETE'
8  * datos    : objeto JS para POST/PUT (opcional)
9  */
10 async function pedirAPI(url, metodo = 'GET', datos = null) {
11   // 1. Configurar la peticion
12   const opciones = {
13     method: metodo,
14     headers: { 'Content-Type': 'application/json' }
15   };
16
17   // 2. Agregar body solo en POST y PUT
18   if (datos && (metodo === 'POST' || metodo === 'PUT')) {
19     opciones.body = JSON.stringify(datos);
20   }
21
22   // 3. Realizar la peticion HTTP
23   const respuesta = await fetch(url, opciones);
24
25   // 4. Convertir la respuesta JSON a objeto JavaScript
26   const json = await respuesta.json();
27
28   // 5. Si el servidor devolvio error (4xx, 5xx) -> excepcion
29   if (!respuesta.ok) {
30     throw new Error(json.error || 'Error HTTP ' + respuesta.status);
31   }
32
33   return json;
34 }
```

8.3. Leer y mostrar cultivos

```
1 async function cargarCultivos() {
2   try {
3     // Llamar a la API: GET /api/cultivos
4     const data = await pedirAPI(BASE_URL + '/cultivos');
5     const tbody = document.getElementById('tabla-cuerpo');
6
7     tbody.innerHTML = data.cultivos.map(function(c) {
8       return '<tr>'
9         + '<td>' + c.id + '</td>'
10        + '<td>' + c.nombre + '</td>'
11        + '<td>' + c.variedad + '</td>'
12        + '<td>' + c.fecha_siembra + '</td>'
13        + '<td>' + c.area_hectareas + ' ha</td>'
14        + '<td>' + c.estado + '</td>'
15        + '<td>'
16        + '<button onclick="editarCultivo(' + c.id + ')">Editar</button>'
17        + '<button onclick="eliminarCultivo(' + c.id + ')">Eliminar</button>'
18      + '</td>'
19      + '</tr>';
20     });
21   } catch (error) {
22     console.error('Error al cargar cultivos: ', error);
23   }
24 }
```

```

18     + '</td>'
19     + '</tr>';
20   }).join('');
21
22   } catch (error) {
23     console.error('Error al cargar:', error.message);
24     alert('No se pudo conectar con la API');
25   }
26 }
27
28 cargarCultivos(); // Ejecutar al cargar la pagina

```

8.4. Crear, editar y eliminar

```

1  // -- CREAR un nuevo cultivo (POST) -----
2  async function crearCultivo() {
3    const nuevo = {
4      nombre:      document.getElementById('campo-nombre').value,
5      variedad:    document.getElementById('campo-variedad').value,
6      fecha_siembra: document.getElementById('campo-siembra').value,
7      parcela_id:  parseInt(document.getElementById('campo-parcela').value),
8      area_hectareas: parseFloat(document.getElementById('campo-area').value),
9      estado:      'germinacion'
10   };
11   try {
12     const res = await pedirAPI(BASE_URL + '/cultivos', 'POST', nuevo);
13     alert('Cultivo creado con ID: ' + res.cultivo.id);
14     cargarCultivos();
15   } catch (e) { alert('Error: ' + e.message); }
16 }
17
18 // -- ACTUALIZAR un cultivo existente (PUT) -----
19 async function actualizarEstado(id, nuevoEstado) {
20   try {
21     await pedirAPI(BASE_URL + '/cultivos/' + id,
22                   'PUT', { estado: nuevoEstado });
23     cargarCultivos();
24   } catch (e) { alert('Error al actualizar: ' + e.message); }
25 }
26
27 // -- ELIMINAR un cultivo (DELETE) -----
28 async function eliminarCultivo(id) {
29   if (!confirm('Eliminar cultivo #' + id + '?')) return;
30   try {
31     await pedirAPI(BASE_URL + '/cultivos/' + id, 'DELETE');
32     cargarCultivos();
33   } catch (e) { alert('Error al eliminar: ' + e.message); }
34 }

```

9. Migración a PostgreSQL

Cuando el proyecto crezca puedes reemplazar los archivos JSON por una base de datos real. La **Tabla 4** resume los cambios necesarios, que son mínimos gracias a la

separacion en capas del proyecto.

Con JSON (ahora)	Con PostgreSQL (produccion)
<code>fs.readFileSync(DB_PATH)</code>	<code>await pool.query('SELECT * FROM...')</code>
<code>JSON.parse(contenido)</code>	Resultados en <code>resultado.rows</code>
<code>fs.writeFileSync(...)</code>	<code>await pool.query('INSERT INTO...')</code>
Autogenerar ID con <code>Math.max</code>	<code>SERIAL</code> o <code>GENERATED ALWAYS AS IDENTITY</code>

Cuadro 4: Cambios necesarios para migrar de JSON a PostgreSQL.

```

1 // Ejemplo: GET /api/cultivos con PostgreSQL
2 const { Pool } = require('pg');
3 const pool = new Pool({
4   host: 'localhost', port: 5432,
5   database: 'agro_db',
6   user: 'postgres', password: 'mi_password'
7 });
8
9 router.get('/', async (req, res) => {
10   try {
11     const resultado = await pool.query(
12       'SELECT * FROM cultivos ORDER BY id'
13     );
14     res.json({
15       total: resultado.rowCount,
16       cultivos: resultado.rows
17     });
18   } catch (e) { res.status(500).json({ error: e.message }); }
19 });

```

10. Resumen y Proximos Pasos

10.1. Lo que aprendiste en este tutorial

- **Arquitectura REST** (Figura 1): separacion en capas Cliente → API → Datos.
- **Verbos HTTP** (Tabla 1): GET, POST, PUT, DELETE.
- **Bases de datos de archivo** (Tabla 2): JSON y XML como almacenamiento simple.
- **Node.js + Express**: servidor HTTP con rutas parametrizadas (Figura 3).
- **Middleware**: `cors()`, `express.json()`.
- **fetch()** (Figura 4): peticiones asincronas con `async/await`.
- **Manejo de errores**: codigos HTTP y bloques `try/catch`.

- **Migracion a PostgreSQL** (Tabla 4): que lineas cambiar cuando el proyecto crezca.

10.2. Proximos pasos sugeridos

1. **Autenticacion:** agrega JWT (jsonwebtoken) para proteger los endpoints.
2. **Validacion:** usa joi o express-validator para validar el body.
3. **Base de datos real:** migra con pg o usa Sequelize o Prisma.
4. **Variables de entorno:** mueve las credenciales a .env con dotenv.
5. **Despliegue:** publica tu API en Railway, Render o Heroku.

► Importante

Nunca subas contraseñas, claves API o credenciales de base de datos a GitHub. Siempre usa archivos .env y agragalos al .gitignore.